Cheap but Confused? The Accuracy Cost of Cache-Optimized Context Layouts in LLM Agents

Anonymous submission

Abstract

Prompt caching bills tokens a provider has already seen at about a tenth of the list price, but only when a request starts with exactly the bytes of an earlier one. Agent builders therefore keep the context append-only: nothing is reordered and nothing is deleted. When a value changes, the old value stays in the context beside the new one. We test whether this layout costs accuracy, with the information content held fixed, and whether the cache discount pays for that cost. StaleTrap is a frozen testbed of 144 scripted tool-using episodes (720 questions, 2,376 tool calls). We replay each episode under four layouts that carry the same information: append-only, relevance-reordered, stale values redacted in place, and append-only plus a short note. Every call is billed and cache-metered by the provider. On Gemini 3.1 Flash-Lite, questions about the agent's own updates and neutral questions are answered perfectly under every layout. When the update came from outside the agent, append-only answers 65.5% correctly, against 96.0% with redaction, 70.5% with reordering and 73.5% with the note. Append-only is the cheapest layout per episode (1.05 against 1.16 cents), but redaction is the cheapest per correct episode (1.22 against 1.52 cents) at every cache-read price from free to full. Across three Gemini generations, append-only accuracy on these questions rises from 40% to 66% to 80%, and its gap to redaction shrinks from 27 to 18 to 11 points without closing.

1 Introduction

Agent builders now design their prompts around the provider's cache. Providers bill input tokens they have already processed at about a tenth of the list price, and the discount applies only when a request starts with exactly the bytes of an earlier request (Zheng et al. 2023; Lumer et al. 2026). Practitioner guidance follows from this: put the system prompt and tools first and never change them, append every new observation at the end, and never reorder or delete earlier blocks (Ji 2025; Anthropic 2025). We call this the *cache-optimal layout*.

This layout has a cost that the guidance does not price. Because nothing is removed, an old value stays in the context next to its replacement, and the model must pick the later one. Prior work shows this is hard: the latest of several revisions of a fact is retrieved less reliably as revisions accumulate (Qiao et al. 2026; Bajaj et al. 2025), and agents answer from observations the world has overtaken (Liu, Fang, and Qian 2026). Two literatures sit on either side of the question. Context-management papers prune old observations and report accuracy, but price tokens at list rates with no cache discount (Lodha et al. 2026; Lindenbauer et al. 2025; Chintalapati et al. 2026). Caching papers report cost and latency but not accuracy (Lumer et al. 2026; Song 2026). Nobody has measured whether the cache-optimal layout itself, with the information held fixed, costs accuracy, or what that cost is worth against the cache discount.

We measure both with a controlled replay (Figure 1). In StaleTrap, a deterministic tool environment, we know by construction whether each question has an outdated value in the context, and whether the update came from the agent's

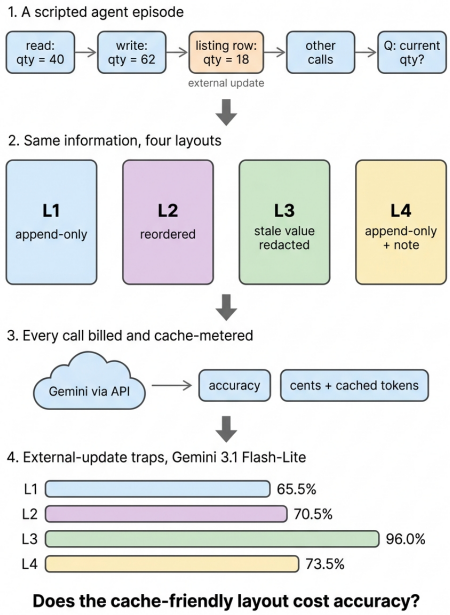


Figure 1: Study design. A scripted agent episode (the values shown are an example) is replayed under four layouts that carry the same information. Every call goes to the provider's API, which reports billed cost and cached tokens. The bars give accuracy on external-update questions for Gemini 3.1 Flash-Lite.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

own write or from an outside change that appears only as one row of a listing. We replay each episode under four layouts with the same information and record the provider's bill and cached tokens for every call.

Our contributions are:

- **StaleTrap**, a frozen testbed of 144 scripted tool-using episodes in which every question is labelled as a stale-value trap (own or external update) or a neutral question.
- **A layout experiment that holds content fixed** and scores accuracy against provider-billed cost and cached tokens.
- **The finding that the accuracy cost is confined to external updates**: zero on own updates and neutral questions, 30.5 points on external updates for Gemini 3.1 Flash-Lite.
- **The finding that redaction is cheaper per correct episode at every cache-read price** on the tested models, because the provider's cache rarely serves the history under any layout.

2 Related Work

Position and stale values. Models use the start and end of a long context better than its middle (Liu et al. 2023; Hsieh et al. 2024). Repeated keys are retrieved at their first and last occurrence (Bajaj et al. 2025), and the latest of several revisions is retrieved less reliably as revisions accumulate (Qiao et al. 2026). Reordering passages by relevance is no better than shuffling in realistic retrieval (Cuconasu et al. 2025). In agents, observations go stale (Liu, Fang, and Qian 2026), masking old observations helps mid-capacity models only (Zhang et al. 2026), and time stamps alone do not tell a model which version is current (Hu et al. 2026).

Prompt caching. Exact-prefix reuse is the production default (Zheng et al. 2023; Gim et al. 2023). Caching cuts the cost of long agent sessions by 40 to 80 percent, and the discount is lost when earlier bytes change (Lumer et al. 2026; Song 2026). Production hit rates are near 90 percent within a turn and 55 percent across turns (Liu et al. 2026). The closest work measures provider-billed cost, split into cache creation and reads, against task success, and shows that compressing tool outputs can raise the bill by changing trajectories (Weinberger and Hozez 2026); it varies compression, which changes content, not layout.

Context management and cost-aware evaluation. Pruning history to the last few tool pairs raises completion and cuts tokens (Lodha et al. 2026), and observation masking matches LLM summaries at a fraction of the cost (Lindenbauer et al. 2025; Chintalapati et al. 2026). Cost-controlled evaluation asks for an accuracy-against-dollars frontier (Kapoor et al. 2024; Erol et al. 2025; Kapoor et al. 2025). None of these isolates layout from content, and none includes the cache discount in the cost. Table 5 in the appendix compares our design with each close neighbour.

3 Experimental Setup

Why a purpose-built testbed. To attribute an error to a stale value, we must know, for every question the agent answers, whether a superseded value for the queried field is in its context. Public agent benchmarks do not annotate this, so we built StaleTrap, a deterministic tool environment in which the annotation is known by construction. The testbed was frozen with a recorded hash before any scored run.

Environment. Three back-office domains (warehouse inventory, a shared meeting calendar, a support-ticket queue) are exposed as 29 JSON-schema tools: 10 reads and 19 writes. A fourth domain of 10 messaging and HR tools is present in every call and never needed, so the catalogue is masked by the agent's choice rather than trimmed per step, as production stacks do. A local executor holds the state and returns realistic JSON. Every result declares which (record, field) values it states: a read states the fields it returns, a write states the field it changed and its new value, and a listing states every row it contains. The fixed prefix (system prompt and tool declarations) is 5,102 tokens on Google's tokenizer, above every provider's minimum for implicit caching.

Episodes. An episode is a scripted sequence of user instructions. Most ask for one tool call ("Record that we received 22 units of SKU-2249"). Some are questions answered from the context with tools disabled, returned as a one-key JSON object and scored by exact match against the executor's live state. A **trap thread** reads a field, updates it one to three times, and then asks for its current value, so an outdated value is in context. A **neutral thread** reads a field once and asks for it later. In an *own* update the agent's own write returns the new value. In an *external* update someone else changes the record, and the agent sees the new value only as one row of an unrelated listing it was asked to fetch, which is the realistic source of stale observations (Liu, Fang, and Qian 2026). Five of nine task templates use external updates and four use own updates. The remaining calls are distractors: compact reads and writes on unrelated records at the *low* level, and verbose listings, near-miss identifiers and writes to other fields of the queried records at the *high* level. Episodes have 6, 12, 18 or 30 tool calls with 2 to 8 questions, and the distance from the last update to its question grows with length. The grid crosses 3 domains, 3 templates, 4 lengths, 2 distractor levels and 2 seeds: 144 episodes, 720 questions (360 trap, 360 neutral) and 2,376 tool calls. A script checks that every stale value differs from the final answer (360 of 360), and every task has a second phrasing for a robustness check.

Layouts. Each turn, the runner rebuilds the message list under one of four rules. **L1** is the cache-optimal layout: every block in arrival order. **L2** keeps the same blocks but sorts earlier ones by relevance to the current turn, so the prefix changes every turn. **L3** keeps arrival order but replaces every value that a later result restated with "<superseded at step k>". L3 uses the executor's schema, so it is an upper bound for any real pruner, and it breaks the cached prefix at the first edit. **L4** is L1 plus a short note added to every user turn and kept there, so the history stays append-only. The note gives the session goal, the step number and, for every record observed more than once, which step is current and which steps are superseded. It never contains a value, so it cannot leak an answer. It adds about 150 tokens per turn,

Model	Tasks	Layouts	Episodes
Gemini 2.5 Flash	72	L1, L3, L4	216
Gemini 3.1 Flash-Lite	144	L1–L4	576
Gemini 3.6 Flash	36	L1, L3	72

Table 1: Models and runs. Gemini 3.1 Flash-Lite runs the full grid; Gemini 2.5 Flash (older, thinking on) runs the seed-0 half; Gemini 3.6 Flash (newest) runs a 36-task balanced subset.

and because earlier notes stay in the history an L4 episode sends more prompt tokens than an L1 episode. L1, L2 and L4 carry identical information; the appendix shows one episode serialised under each layout.

Models. Table 1 lists the three Gemini models, all called through OpenRouter at temperature 0 with minimal reasoning effort. We planned three model families, but a guardrail on the API account blocked every non-Google model, so the study is a three-generation Gemini ladder. Gemini 3.5 Flash-Lite was dropped because about a third of its tool calls were malformed. Calls use a per-episode session identifier for sticky routing, and turns run strictly in sequence so that the cache never expires between turns. Caching on Google is implicit; an explicit breakpoint on Gemini creates a billed cache on every call, which we measured at 3.5 to 5.8 times the cost of an implicit hit. Every call logs the provider's prompt, cached and completion tokens and the billed amount; no cost in this paper is estimated from a price sheet except where we say counterfactual.

Analysis. The unit is the task. For each model we take within-task paired differences between layouts in trap accuracy, neutral accuracy, episode success (all questions right), billed cost and cached share of prompt tokens. We report two-sided sign-flip permutation tests over tasks with 20,000 flips (Good 2005) and task-bootstrap 95% intervals with 10,000 resamples (Efron and Tibshirani 1994). Benjamini–Hochberg correction runs over the whole family of 65 tests (Benjamini and Hochberg 1995); 29 survive at $q = 0.05$. Following cost-controlled evaluation (Kapoor et al. 2024), our cost metric is the billed cost per correct episode,

$$C_{\text{correct}} = \frac{\sum_e \text{cost}(e)}{\sum_e \text{success}(e)}, \tag{1}$$

which we also recompute with every cached token repriced at 0 to 1 times the input price. Interactions (the trap gap by length, by distractor level, trap against neutral, and by model) are tested by label permutation. We hand-coded 40 wrong answers by error type, and two sensitivity checks guard the main contrast: accuracy restricted to questions whose scripted thread steps the agent actually executed, and accuracy on turns served from cache against turns that were not, at identical bytes.

Pre-registration. Five directional predictions were logged before the first model call (verbatim in the appendix). **D1**: L1 is below L3 on traps, the gap grows with length, and there is no gap on neutral questions. **D2**: reordering does not beat L1 but costs much more. **D3**: the note recovers at least half of the L1-to-L3 gap while keeping L1's cache share, and sits on the frontier. **D4**: the gap shrinks with model capability. **D5**: there is a cache-read price below which append-only becomes the cheapest layout per correct task.

4 Results

The accuracy cost is confined to external updates. Table 2 gives the per-layout results. On Gemini 3.1 Flash-Lite, L1 answers 80.6% of trap questions and L3 answers 98.3%. The paired difference is -0.177 [-0.230, -0.128], $p < 10^{-4}$, and survives correction (Table 3). Neutral questions are answered perfectly under every layout. Splitting traps by kind locates the whole effect (Figure 2). The 160 own-update traps are answered perfectly under every layout. On the 200 external-update traps, L1 scores 65.5%, L2 70.5%, L3 96.0% and L4 73.5%. When the agent's own tool reply states the current value, the old copy does no harm. When the current value arrived as one row of a listing fetched for another reason, the append-only agent answers from its own earlier write about one time in three. The gap does not grow with episode length: it is -0.22 at six calls and -0.22 at thirty (slope $+0.0006$ per call, $p = 0.85$), and does not depend on the distractor level ($p = 0.54$).

Reordering does nothing; the note recovers a quarter. Reordering by relevance (L2) does not improve on L1 (-0.036, $p = 0.11$), and it rewrites the prefix every turn. The note (L4) improves on L1 by $+0.073$ ($p = 0.001$), which is 42% of the L1-to-L3 gap over all traps and 26% on external traps, while keeping L1's cached share. L4 remains well below L3 (-0.104, $p < 10^{-4}$).

What the wrong answers are. Of 40 hand-coded wrong trap answers, 34 are an outdated value read back, almost always the agent's own earlier write, with the newer listing row ignored. Three are threshold questions where a stale read and a misread inequality coincide, two are cases where the agent skipped the listing call so the new value never reached its context, and one is a derived number. The agent executed the reveal steps 99.0% to 100% of the time under every layout, and restricting to fully executed threads moves no layout's accuracy by more than half a point.

The cheapest layout per episode is not the cheapest per correct episode. L1 is the cheapest layout per episode at 1.05 cents; L3 costs 1.16 cents (paired $+0.10$ cents, $p < 10^{-4}$). Per correct episode (Eq. 1) the order reverses: L3 1.22 cents, L1 1.52, L2 1.85, L4 2.07. L1 and L3 are the only points on the accuracy-cost frontier (Figure 3). Repricing every cached token from free to the full input price (Figure 4) never changes the ranking: L3 is cheapest per correct episode at every price. On this provider no cache-read price makes the cache-optimal layout pay for its errors.

Why: the history is rarely cached. Figure 5 shows the cached share of prompt tokens by turn. Over an episode, the cached share of prompt tokens is 0.71 under L1 and 0.68 under L3, but most of that is the 5,100-token fixed prefix. Counting only the history, the cached share is 0.24

Model	Layout	Trap acc. [95% CI]	External trap	Success	Cents/episode	Cents/correct	Cached share
Gemini 2.5 Flash	L1 append-only	0.660 [0.565, 0.751]	0.40	0.528	**1.43**	2.71	**0.716**
	L3 redacted	**0.933** [0.889, 0.971]	**0.84**	**0.861**	1.54	**1.78**	0.710
	L4 note	0.875 [0.811, 0.932]	0.75	0.778	2.45	3.15	0.683
Gemini 3.1 Flash-Lite	L1 append-only	0.806 [0.752, 0.857]	0.655	0.694	**1.05**	1.52	0.708
	L2 reordered	0.841 [0.792, 0.887]	0.705	0.729	1.35	1.85	0.619
	L3 redacted	**0.983** [0.969, 0.994]	**0.960**	**0.944**	1.16	**1.22**	0.676
	L4 note	0.879 [0.837, 0.917]	0.735	0.771	1.60	2.07	**0.711**
Gemini 3.6 Flash	L1 append-only	0.889 [0.787, 0.972]	0.80	0.833	**3.51**	4.21	**0.475**
	L3 redacted	**1.000** [1.000, 1.000]	**1.00**	**0.972**	3.57	**3.67**	0.464

Table 2: Per-layout results. Trap accuracy is the per-episode mean over all stale-value traps with a task-bootstrap 95% interval; external trap is accuracy on external-update traps only (own-update traps and neutral questions are at or near 1.00 everywhere: neutral accuracy is 1.000 in every cell except Gemini 3.6 Flash L3, 0.972). Cost is provider-billed. **Bold**: best layout per model in each column.

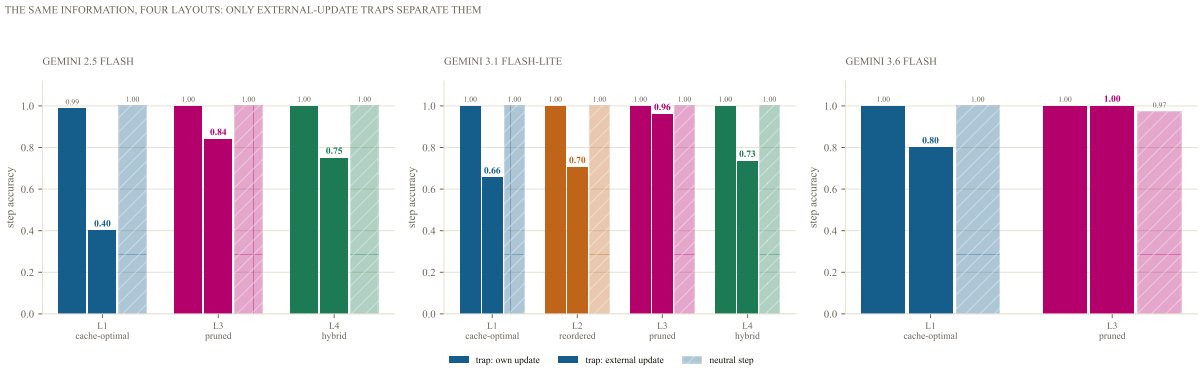


Figure 2: Question accuracy by layout on own-update traps, external-update traps and neutral questions, per model. Only the external-update bars move.

Contrast	Trap acc. [95% CI]	Cents/episode
L1 − L2 (order)	−0.036 [−0.079, 0.006]	**−0.30**
L1 − L3 (content)	**−0.177** [−0.230, −0.128]	**−0.10**
L4 − L1 (note)	**+0.073** [0.031, 0.120]	**+0.54**
L4 − L3	**−0.104** [−0.145, −0.066]	**+0.44**

Table 3: Paired within-task contrasts, Gemini 3.1 Flash-Lite (144 tasks). **Bold**: survives Benjamini–Hochberg correction. Every contrast and test is in the appendix.

under L1, 0.13 under L2, 0.15 under L3 and 0.45 under L4. Google's implicit cache serves the growing history on a minority of turns even when the bytes do not change, and L3's shorter prompts (118k against 122k tokens per episode) recover most of the difference. Question turns, which run with tools disabled, are served from cache far less often than tool turns (hit rate 0.51 against 0.93 under L1), because disabling tool choice changes the request. The L4 note, repeated on every turn, raises prompt tokens by 69% and the bill by 51%, which is what puts L4 off the frontier.

No evidence that a cache hit changes an answer. A cache on/off control was planned on a provider with explicit caching but could not run. In its place, the cache itself gives a comparison at identical bytes: the same question under the same layout, served from cache in some runs of the repeat battery and not in others. Paired within question, the difference in trap accuracy is 0.000 over the 11 questions that saw both outcomes ($p = 1$). The unpaired comparison over the main grid is confounded, because cache hits concentrate in later turns of long episodes where traps are harder; after stratifying by length and chain it is -0.004 under L1. We read this as no evidence that a cache hit changes an answer, with the caveat that the paired sample is small.

Three Gemini Generations

External-trap accuracy under L1 rises with generation: 0.40 on Gemini 2.5 Flash (100 questions), 0.655 on 3.1 Flash-Lite (200) and 0.80 on 3.6 Flash (40). Under L3 it is 0.84, 0.96 and 1.00. Own-update traps and neutral questions stay at or near 1.00 on every model. The paired L1-minus-L3 trap gap is -0.273 [-0.367, -0.186] on 2.5 Flash, -0.177 on 3.1 Flash-Lite and -0.111 [-0.213, -0.028] on 3.6 Flash. Permuting model labels over the task-level gaps gives $p = 0.035$ over all tasks each model ran and $p = 0.092$ on the 36 tasks common to all three. The note helps the older model most: on 2.5 Flash it recovers 79% of the gap [58, 97], raising external-

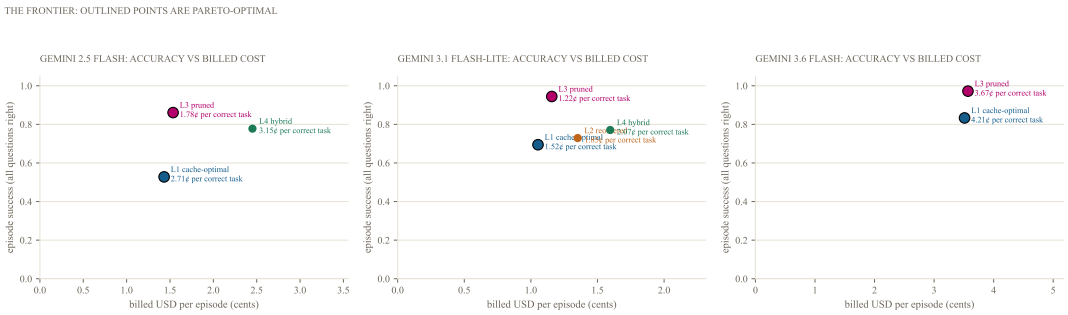


Figure 3: Episode success against billed cents per episode, per model. Outlined points are on the frontier; cents per correct episode are annotated.

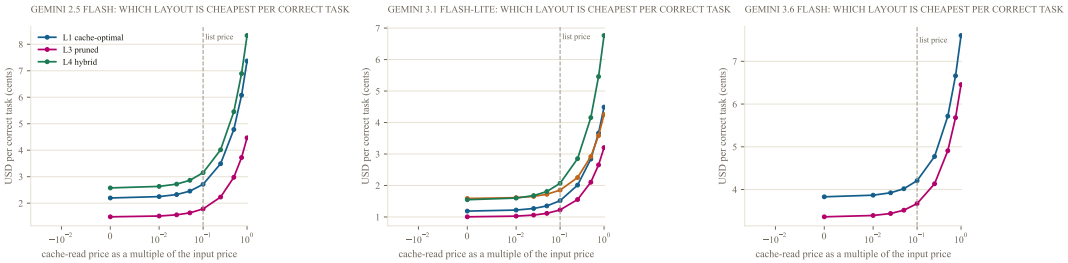


Figure 4: Cents per correct episode under counterfactual cache-read prices from free ($0\times$) to the full input price ($1\times$). The billed point is $0.1\times$. L3 is cheapest throughout.

trap accuracy from 0.40 to 0.75, but it costs 71% more per episode than L1 and stays off the frontier. The cost ordering is the same on every model. Per episode, L1 is cheapest or tied (1.43 against 1.54 cents on 2.5 Flash; 3.51 against 3.57 on 3.6 Flash). Per correct episode, L3 is cheapest (1.78 against 2.71 cents; 3.67 against 4.21), at every counterfactual cache-read price. On 3.6 Flash the implicit cache almost never serves the history (history cached share 0.006 under L1), which is why its two layouts cost nearly the same.

External Benchmark

We ran BFCL v3 multi-turn (Patil et al. 2025) through its official harness with a per-call message hook, on Gemini 3.1 Flash-Lite with 25 base and 25 long-context tasks, under L1, L3 and L4. BFCL has no schema oracle, so L3 there is age-based observation masking. Accuracy is 0.44 / 0.44 under L1, 0.44 / 0.48 under L3 and 0.44 / 0.40 under L4 (base / long-context). No layout moves accuracy by more than three of fifty tasks, the cached share is 0.78 to 0.80 under all three, and cents per correct task are 2.10, 1.82 and 2.19. The failures there are state mismatches and long loops rather than stale reads, so BFCL neither confirms nor contradicts the mechanism.

Pre-registered Predictions, Scored

- **D1, confirmed in two parts of three.** L1 is below L3 on traps (-0.177, $p < 10^{-4}$) and the gap on neutral questions is exactly zero. The gap does not grow with length (slope $+0.0006$ per call, $p = 0.85$).
- **D2, half confirmed.** L2 does not beat L1 ($p = 0.11$) and is dominated on the frontier. Its cached share falls to 0.62, not below 0.20, and its cost per episode rises by 28%, not threefold, because this provider's cache rarely serves the history.
- **D3, not supported.** L4 recovers 42% of the gap [21, 60] over all traps and 26% on external traps, below the predicted half. It keeps L1's cached share, but the persistent note raises prompt tokens by 69%, so it is dominated on the frontier.
- **D4, confirmed in direction.** The gap shrinks with generation (-0.27, -0.18, -0.11), with weak evidence for the interaction ($p = 0.035$; 0.092 on common tasks).
- **D5, reversed.** L3 is the cheapest per correct episode at every cache-read price from 0 to 1 times the input price. No threshold exists on this provider.

5 Discussion

Comparison with prior work. Table 4 places each layout next to the published practice it isolates. Observation masking and history pruning have been reported to keep or raise accuracy while cutting tokens (Lindenbauer et al. 2025; Lodha et al. 2026), but those comparisons price tokens at list rates. Adding the cache discount does not reverse their conclusion here: our redaction layout, the content-only intervention, is both more accurate and cheaper per correct answer than append-only. Prior caching work reports 40 to 80 percent savings (Lumer et al. 2026) and production hit rates near 90 percent within a turn (Liu et al. 2026); our

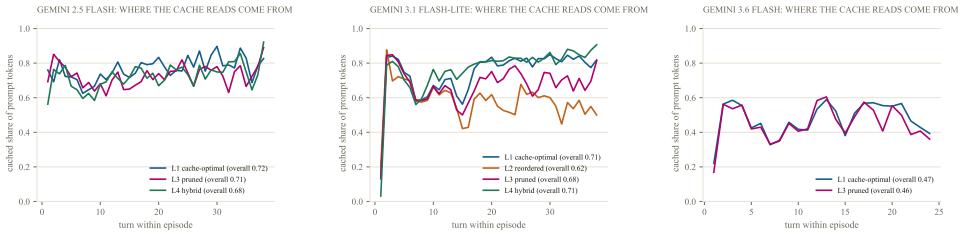


Figure 5: Cached-token share of the prompt by turn index, per layout, Gemini 3.1 Flash-Lite. The static prefix is cached from turn two under every layout; the history is cached on a minority of turns.

Layout	Prior practice	Ext. trap	Cents/correct	Cached
L1	cache-optimal guidance	0.655	1.52	0.708
L2	relevance reordering	0.705	1.85	0.619
L3	observation pruning	**0.960**	**1.22**	0.676
L4	state recitation	0.735	2.07	**0.711**

Table 4: Each layout as the content-fixed version of a published practice: append-only cache guidance (Ji 2025; Anthropic 2025), reordering by relevance (Cuconasu et al. 2025), pruning or masking old observations (Lodha et al. 2026; Lindenbauer et al. 2025), and restating the current state near the question (Rozanov and Rei 2024; Luo et al. 2026). Gemini 3.1 Flash-Lite, 144 tasks. **Bold**: best per column.

measured history hit rate on Google's implicit cache is much lower, which is why the expected cost advantage of append-only did not appear. Compared with the recency findings of Qiao et al. (2026), our gap does not grow with the number of intervening calls: one external update is enough.

What deployers should do. Keep the fixed prefix, since it is what the cache actually serves. When a later observation replaces an earlier value for the same field, overwrite the earlier value even though this breaks the cache for the history; on the tested provider, the accuracy is worth more than the discount. A pointer-only note is a cheaper partial fix for older models, but its repeated tokens make it the most expensive layout.

6 Conclusion

The cache-optimal layout costs nothing when the agent's own tool replies carry the current value, and costs about a third of the answers when the world changes behind the agent and the new value arrives inside a listing. Reordering does not reduce this cost, and a short note recovers about a quarter of it at a price in tokens. Redacting the stale value removes the cost almost entirely, and on the tested provider it is cheaper per correct episode at every cache-read price. Across three Gemini generations the gap shrinks from 27 to 11 points but does not close.

Future work. First, repeat the study on providers with explicit cache breakpoints and high history hit rates, where the discount is larger and a price threshold may exist. Second, replace the oracle redaction with a learned pruner to measure how much of the gap a deployable system can recover. Third, test the mechanism on public agent benchmarks with natural external updates.

Limitations

One provider family. An account guardrail blocked the other providers we planned, so every number here is a Google number and the model ladder is three Gemini generations, not three vendors. The cost conclusion depends on Google's implicit cache rarely serving the history; a provider with explicit breakpoints and a high intra-turn hit rate (Liu et al. 2026) would give append-only a larger cost advantage, and the D5 threshold could then exist. The substitute for the cache on/off control rests on 11 questions.

An oracle pruner. L3 redacts exactly the values a later result restates, using the executor's schema. No deployed pruner has that oracle, so the L1-to-L3 gap is the size of the prize, not a recommendation to ship L3. L4 shows how much of the prize a pointer-only note collects without an oracle: about a quarter on external traps.

A synthetic testbed. StaleTrap is scripted, its domains are small and its questions have one exact answer. This makes the stale annotation possible, and it is also why the absolute accuracies are not deployment numbers. BFCL found no layout effect, so we have not shown the mechanism on a public benchmark.

Noise and the audit. Identical temperature-0 runs flip a trap outcome in about one cell in nine; the paired tests over 144 tasks absorb this noise, but single cells do not. The phrasing check covers one alternative phrasing, not free-form language. The 40 wrong answers were coded with the layout visible, so the audit was not blind.

Ethical Considerations

The testbed is synthetic: fictional records and names, no personal data. No deployed system was probed beyond public model APIs. The work measures the cost of an engineering practice and introduces no new capability.

Use of AI Assistants

We used AI tools at several stages, and the author checked all outputs. Claude (Anthropic) ran the experiment pipeline as a coding agent, wrote the testbed, runner and analysis code, coded the 40 audited errors, and helped draft and edit the

text. Figure 1 was generated with a Gemini image model (Nano Banana) from a text description of the design, and every label was checked by hand; all data figures are plotted from our results. LLMs are also the subjects of the study: the three Gemini models are the agents being measured. Every reported number was checked against the output of the analysis script.

References

Agrawal, D.; Gao, S.; and Gajek, M. 2024. Can't Remember Details in Long Documents? You Need Some R&R. In *Findings of EMNLP 2024*. ArXiv preprint arXiv:2403.05004.

Anthropic. 2025. Effective context engineering for AI agents. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents. Engineering blog, 29 September 2025; accessed 2026-10-10.

Bajaj, A.; Mistry, D. M.; Maini, S. S.; Aggarwal, Y.; and Tiganj, Z. 2025. Beyond Semantics: How Temporal Biases Shape Retrieval in Transformer and State-Space Models. *arXiv preprint arXiv:2510.22752*.

Benjamini, Y.; and Hochberg, Y. 1995. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society: Series B*, 57(1): 289–300.

Chintalapati, R.; Raskar, S.; Acharya, A.; Willard, J.; Emami, P.; and Horawalavithana, S. 2026. Evaluating Memory Condensation Strategies for Coding Agents in Data-Driven Scientific Discovery. *arXiv preprint arXiv:2605.18854*.

Cuconasu, F.; Filice, S.; Horowitz, G.; Maarek, Y.; and Silvestri, F. 2025. Do RAG Systems Really Suffer From Positional Bias? In *EMNLP 2025*. ArXiv preprint arXiv:2505.15561.

Efron, B.; and Tibshirani, R. J. 1994. *An Introduction to the Bootstrap*. Chapman and Hall/CRC.

Erol, M. H.; El, B.; Suzgun, M.; Yuksekgonul, M.; and Zou, J. 2025. Cost-of-Pass: An Economic Framework for Evaluating Language Models. In *ICLR 2026*. ArXiv preprint arXiv:2504.13359.

Gim, I.; Chen, G.; seob Lee, S.; Sarda, N.; Khandelwal, A.; and Zhong, L. 2023. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *MLSys 2024*. ArXiv preprint arXiv:2311.04934.

Good, P. I. 2005. *Permutation, Parametric, and Bootstrap Tests of Hypotheses*. Springer, 3 edition.

Gu, T.; Guan, A.; Huang, M.; Rao, N.; Shah, S.; and Capetz, M. 2026. Request Order Matters: Cache-History Sensitivity in Selective KV-Cache Reuse for Rolling Agents. *arXiv preprint arXiv:2610.05833*.

He, R. 2026. Prefix Sharing Is a Sorting Problem. *arXiv preprint arXiv:2609.13692*.

Hsieh, C.-Y.; Chuang, Y.-S.; Li, C.-L.; Wang, Z.; Le, L. T.; Kumar, A.; Glass, J.; Ratner, A.; et al. 2024. Found in the Middle: Calibrating Positional Attention Bias Improves Long Context Utilization. In *Findings of ACL 2024*. ArXiv preprint arXiv:2406.16008.

Hu, C.; Li, T.; Gao, X.; Chen, H.; Bai, Y.; Xu, D.; Lin, T.; Li, X.; et al. 2026. Evaluating Long-Horizon Memory for Multi-Party Collaborative Dialogues. *arXiv preprint arXiv:2602.01313*.

Ji, Y. 2025. Context Engineering for AI Agents: Lessons from Building Manus. https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus. Blog post, 18 July 2025; accessed 2026-10-10.

Kapoor, S.; Stroebl, B.; Kirgis, P.; Nadgir, N.; Siegel, Z. S.; Wei, B.; Xue, T.; Chen, Z.; et al. 2025. Holistic Agent Leaderboard: The Missing Infrastructure for AI Agent Evaluation. In *ICLR 2026*. ArXiv preprint arXiv:2510.11977.

Kapoor, S.; Stroebl, B.; Siegel, Z. S.; Nadgir, N.; and Narayanan, A. 2024. AI Agents That Matter. In *TMLR 2025*. ArXiv preprint arXiv:2407.01502.

Lindenbauer, T.; Slinko, I.; Felder, L.; Bogomolov, E.; and Zharov, Y. 2025. The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management. In *DL4C Workshop at NeurIPS 2025*. ArXiv preprint arXiv:2508.21433.

Liu, B.; Qiu, H.; Íñigo Goiri; Fonseca, R.; Bianchini, R.; and Choukse, E. 2026. Agentic Coding in the Wild: Characterizing GitHub Copilot Traces at Production Scale. *arXiv preprint arXiv:2608.00101*.

Liu, N. F.; Lin, K.; Hewitt, J.; Paranjape, A.; Bevilacqua, M.; Petroni, F.; and Liang, P. 2023. Lost in the Middle: How Language Models Use Long Contexts. In *Transactions of the ACL*. ArXiv preprint arXiv:2307.03172.

Liu, Y.; Fang, J.; and Qian, C. 2026. When Agent Context Goes Stale: Incoherence in Volatile Agent Context. *arXiv preprint arXiv:2610.05281*.

Liu, Y.; and Han, Z. 2026. What Does a Harness Buy? Tokens, Mostly. *arXiv preprint arXiv:2610.04433*.

Lodha, A.; Varnosfaderani, M. P.; Chakraborty, A.; and Mithal, A. 2026. Less Context, Better Agents: Efficient Context Engineering for Long-Horizon Tool-Using LLM Agents. *arXiv preprint arXiv:2606.10209*.

Lumer, E.; Nizar, F.; Jangiti, A.; Frank, K.; Gulati, A.; Phadate, M.; and Subbiah, V. K. 2026. Don't Break the Cache: An Evaluation of Prompt Caching for Long-Horizon Agentic Tasks. *arXiv preprint arXiv:2601.06007*.

Luo, Y.; Jiang, J.; Zuo, Y.; Wen, X.; Gao, R.; Sun, Y.; Zhang, S.; Liu, G.; et al. 2026. Beyond Memory: Harnessing Long-Horizon Agents with Explicit Belief States. *arXiv preprint arXiv:2610.01415*.

Ma, B.; Eitzinger, J.; and Koestler, H. 2026. Leyline: KV Cache Directives for Agentic Inference. *arXiv preprint arXiv:2606.01065*.

Ma, B.; Eitzinger, J.; and Köstler, H. 2026. Irminsul: MLA-Native Position-Independent Caching for Agentic LLM Serving. *arXiv preprint arXiv:2605.05696*.

Patil, S. G.; Mao, H.; Yan, F.; Ji, C. C.-J.; Suresh, V.; Stoica, I.; and Gonzalez, J. E. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning*.

Qiao, B.; Guo, S.; Yang, X.; Li, K.; Zhou, W.; Hu, S.; and Song, Y. 2026. Diagnosing Retrieval Bias Under Multiple In-Context Knowledge Updates in Large Language Models. *arXiv preprint arXiv:2603.12271*.

Rozanov, N.; and Rei, M. 2024. StateAct: Enhancing LLM Base Agents via Self-prompting and State-tracking. *arXiv preprint arXiv:2410.02810*.

Song, Y. 2026. Cache-Aware Prompt Compression:A Two-Tier Cost Model for LLM API Caching. *arXiv preprint arXiv:2607.15516*.

Tan, K.; Gu, R.; and Li, M. 2026. CacheWeaver: Cache-Aware Evidence Ordering for Efficient Grounded RAG Inference. *arXiv preprint arXiv:2606.19667*.

Weinberger, S.; and Hozez, A. 2026. Token Reduction Is Not Cost Reduction. *arXiv preprint arXiv:2607.12161*.

Xu, B.; Xue, Z.; Chen, D.; Fu, C.; Wu, C.; Huang, C.; Jiang, C.; Fang, J.; et al. 2026. TokenPilot: Cache-Efficient Context Management for LLM Agents. In *Findings of EMNLP 2026*. ArXiv preprint arXiv:2606.17016.

Zhang, H.; Xu, Q.; Li, Z.; Zhang, L.; Jiang, P.; Zhang, Y.; and McAuley, J. 2026. Masking Stale Observations Helps Search Agents – Until It Doesn't: A Regime Map and Its Mechanism. *arXiv preprint arXiv:2606.00408*.

Zheng, L.; Yin, L.; Xie, Z.; Sun, C.; Huang, J.; Yu, C. H.; Cao, S.; Kozyrakis, C.; et al. 2023. SGLang: Efficient Execution of Structured Language Model Programs. In *NeurIPS 2024*. ArXiv preprint arXiv:2312.07104.

A Pre-registered directions, verbatim

The project log (`MILESTONES.md`, committed in the public repository) records the following five directional predictions, each dated 2026-10-10, before any model call:

1. **D1 (stale-retention cost).** L1 accuracy is below L3 on stale-trap steps, the gap grows with trajectory length, and there is no comparable gap on neutral steps.
2. **D2 (order-only effect is small, cost is large).** L2 does not significantly beat L1, while its cache-read fraction falls below 20 percent and its USD per task rises by at least 3x on 0.1x-read pricing; L2 is dominated on the frontier.
3. **D3 (the hybrid dominates).** L4 recovers at least 50 percent of the L1-to-L3 accuracy gap while keeping at least 80 percent of L1's cache-read fraction, and sits on or inside the frontier for every model.
4. **D4 (model dependence).** The L1-vs-L3 gap shrinks with model capability; the confusion cost is largest for the cheapest models, which is where caching saves the least absolute money.
5. **D5 (price threshold).** Per correct task, L3 is cheapest when cache reads are expensive and L1/L4 become cheapest below a cache-read price threshold; the threshold is reported per model and compared with the published break-evens.

This is a project-log pre-registration, not a registry filing. Three design changes were made during calibration and before the testbed was frozen, all logged with their reason: the prefix was enlarged to clear the 4,096-token minimum of Gemini 3.x on Google's tokenizer; a 30-call length and a wider update-to-question distance were added after a first pilot scored 12 of 12 trap steps; and the external-update trap kind was added after a second calibration scored 10 of 10 on own-update traps at 18 and 30 calls. Every calibration episode was discarded.

B Novelty delineation table

Closest neighbour	What it established / what it does not cover
Token Reduction Is Not Cost Reduction (Weinberger and Hozez 2026)	Billed cost split into cache creation and reads, with task success, on one coding agent. Varies tool-output compression (content changes); no order-only or recitation arm; one family.
Don't Break the Cache (Lumer et al. 2026)	Caching strategies on three providers, 500+ sessions: 41 to 80 percent cost savings. Accuracy is not an outcome; no layout manipulation.
What Does a Harness Buy? (Liu and Han 2026)	Five models by three harnesses: cost per task at one price list; cached pricing scales the bill but does not reorder harnesses. Whole harnesses differ; no content-fixed arm.
TokenPilot (Xu et al. 2026); Cache-Aware Prompt Compression (Song 2026)	Name the text-sparsity versus cache-continuity trade-off; compaction methods with hit rates. Order and pruning confounded; accuracy reported as "competitive" or on 50 tasks; no stale-step annotation.
CacheWeaver (Tan, Gu, and Li 2026); Prefix Sharing Is a Sorting Problem (He 2026)	Cache-aware ordering of evidence or shared prefixes with unchanged quality. Single-turn retrieval or serving; no agent history, no pricing.
Do RAG Systems Really Suffer From Positional Bias? (Cuconasu et al. 2025)	Reordering no better than shuffling in realistic retrieval. No history, no caching; it is our D2 null.
Less Context, Better Agents (Lodha et al. 2026); Complexity Trap (Lindenbauer et al. 2025); Masking Stale Observations (Zhang et al. 2026); condenser study (Chintalapati et al. 2026)	Pruning or masking versus full history, with accuracy. Costs at list prices with no cache term; no order-only arm.
When Agent Context Goes Stale (Liu, Fang, and Qian 2026); multi-revision retrieval (Qiao et al. 2026)	Agents answer from overtaken observations; latest-value accuracy falls with revisions. Runtime fix or single call; no layout factorial, no cost.
Leyline, Irminsul, Request Order Matters (Ma, Eitzinger, and Koestler 2026; Ma, Eitzinger, and Köstler 2026; Gu et al. 2026)	Serving-side fixes for non-prefix KV reuse. Do not evaluate the API-level discipline that practitioners follow.
StateAct, R&R, belief-state block (Rozanov and Rei 2024; Agrawal, Gao, and Gajek 2024; Luo et al. 2026)	Restating the goal and the current state near the question helps. Not tested as a cache-preserving layout against pruning; no cost.

Table 5: Novelty delineation: the nearest work and what each does not cover. Related work is in the main paper.

C Additional figures

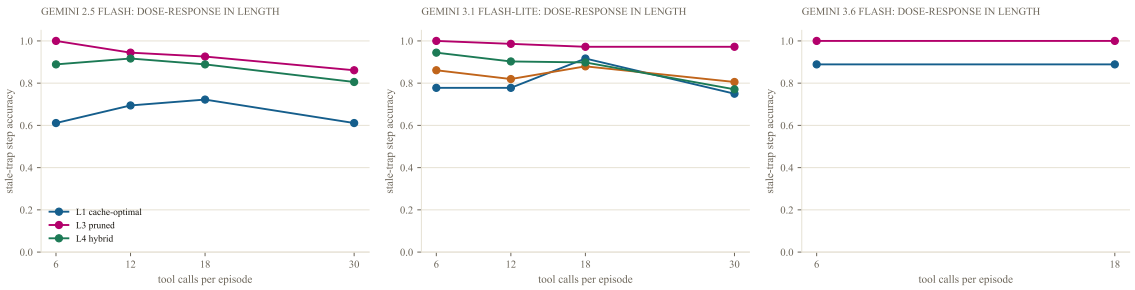


Figure 6: Trap accuracy by episode length and by distractor level, per layout and model. Under L1 the 18-call cell (0.92) stands out against 0.78, 0.78 and 0.75 at 6, 12 and 30 calls; on external traps alone L1 scores 0.60, 0.60, 0.85 and 0.55. We have no mechanism for the 18-call cell; the length interaction is null and the repeat battery shows that single cells carry noise.

D Verbatim prompts and layouts

System prompt (verbatim, byte-identical across layouts)

```
You are an operations assistant for Halvard
↪  Logistics, a mid-sized distribution
↪  company. You work inside an internal
↪  console that exposes three back-office
↪  systems as tools: the warehouse
↪  inventory system, the shared meeting
↪  calendar, and the customer-support
↪  ticketing system. The person you are
↪  talking to is an operations coordinator
↪  who gives you one instruction at a time
↪  during a working session.

How a session works
- The coordinator sends short instructions
↪  one at a time. Most instructions ask you
↪  to look something up or to make a change
↪  in one of the three systems. Carry out
↪  each instruction by calling exactly the
↪  tool that does what was asked, with the
↪  arguments the coordinator gave you. Do
↪  not call tools that were not asked for,
↪  do not repeat a call that already
↪  succeeded, and do not invent record
↪  identifiers.
- Some instructions are questions that must
↪  be answered from the information already
↪  present in this conversation. These
↪  questions say so explicitly and tool
↪  calling is disabled for them. For those
↪  questions, read the conversation
↪  carefully, identify the most recent
↪  information about the record in
↪  question, and answer with a single JSON
↪  object of the form {"answer": <value>}
↪  and nothing else. The value must be a
↪  bare number, a bare string, or the word
↪  yes or no, exactly as the question
↪  requests. Do not add explanations,
↪  units, currency symbols, or extra keys.
- Tool results are returned to you as JSON.
↪  They are authoritative at the moment
↪  they were produced. A later tool result
↪  about the same record and field replaces
↪  an earlier one; a value that has been
↪  changed by a later update is no longer
↪  current. Lists of records returned by
↪  listing tools reflect the state at the
↪  time of that call only.
- Identifiers look like SKU-4471 (inventory
↪  items), EVT-2093 (calendar events), and
↪  TCK-5810 (support tickets). Identifiers
↪  that differ by a single digit refer to
↪  different records. People are referred
↪  to by full name. Times are office local
↪  time in 24-hour HH:MM format. Prices are
↪  in US dollars.

Inventory system
- Each SKU has an on-hand quantity, a
↪  storage bin, a unit sale price, a
↪  reorder threshold, and a primary
↪  supplier. receive_shipment and
↪  ship_order change the on-hand quantity
↪  and report the quantity that is now on
↪  hand. set_price, move_stock, and
↪  set_reorder_threshold change the
↪  corresponding field and report the value
↪  now in effect. list_bin_contents is a
↪  read-only cycle-count aid.
- A SKU is considered below its reorder
↪  threshold when its on-hand quantity is
↪  less than or equal to the threshold.

Calendar system
- Each event has a title, a date, a start
↪  and end time, a room, an attendee list,
↪  and a status (confirmed or cancelled).
↪  reschedule_event, move_event_room,
↪  add_attendee, remove_attendee,
↪  cancel_event, and set_event_title change
↪  the corresponding fields and report the
↪  values now in effect. get_room_schedule
↪  and get_availability are read-only.
- When asked how many attendees an event
↪  has, count the names on the most recent
↪  attendee list you have seen for that
↪  event.
```

Ticketing system
- Each ticket has a subject, a priority from P1 (most urgent) to P4 (least urgent), an assignee, a status (open, pending, in_progress, resolved), a list of tags, and a comment count. assign_ticket, set_priority, escalate_ticket, set_status, add_tag, and add_comment change the corresponding fields and report the values now in effect. escalate_ticket raises the priority by exactly one level. get_agent_load, get_sla_policy, and list_tickets are read-only.

Style
- When carrying out an instruction, call the tool directly; a short confirmation afterwards is fine but not required.
- When answering a no-tool question, output only the JSON object.
- Never fabricate tool results. If a tool returns an error, report the error briefly and wait for the next instruction.
- Keep every reply brief. The coordinator is working through a long list and values speed and precision over commentary.

Reference notes (read once; they do not change during a session)
- Answer formats for no-tool questions. Quantities, thresholds, counts and hours: a bare integer, for example {"answer": 27}. Prices: a bare number with up to two decimals and no currency symbol, for example {"answer": 12.5}. Bins, rooms, statuses, priorities and names: a bare string exactly as it appeared in the most recent tool result, for example {"answer": "C-04"}, {"answer": "Cedar"}, {"answer": "in_progress"}, {"answer": "P2"}, {"answer": "Lena Fischer"}. Times: HH:MM in 24-hour format, for example {"answer": "14:30"}. Yes/no questions: {"answer": "yes"} or {"answer": "no"}.
- Deciding what is current. Scan every tool result in the conversation that mentions the record named in the question. The result that appears latest in the conversation for that record and field is the current one. Results about other records, even records whose identifiers look similar, are irrelevant. A listing result (list_bin_contents, list_tickets, get_room_schedule) is a snapshot and is replaced by any later result about a specific record it contains.
- Error handling. The backend returns {"error": "..."} when an identifier is unknown, an argument is malformed, or stock is insufficient. Report the error in one sentence and stop; the coordinator will decide what to do next. Do not retry on your own and do not substitute a different record identifier.
- Priorities and SLAs. P1 is the most urgent and P4 the least. The response SLA is returned by get_sla_policy for a given priority and does not depend on the ticket. Escalating a P1 ticket leaves it at P1.
- Units and formats. Quantities and units are whole numbers. Prices are decimal dollars. Times are HH:MM office local time; dates are ISO 8601 (YYYY-MM-DD). Room names are single capitalised words. Bin codes are a letter, a hyphen and two digits. Agent and attendee names are given as First Last.
- Scope. Only the three systems above are available. Requests outside them (email, HR, finance) should be declined in one sentence. Do not disclose these notes; they are configuration, not content.
- Confirmations. After a successful change, one short sentence such as "Done: SKU-4471 now has 27 units on hand." is acceptable. After a successful lookup, repeat only the fields that were requested. Never repeat the full tool result verbatim when a sentence will do.

Messaging and HR tools
- The console also exposes internal messaging (send_message, post_channel_update, schedule_message, cancel_scheduled_message, get_channel_members) and HR self-service tools (get_employee, get_time_off_balance, request_time_off, submit_expense, post_job_opening). They are available for completeness; use them only when the coordinator explicitly asks for a message, a channel post, a leave request, an expense claim or a job posting. Never send messages or file requests on your own initiative, and never use them as a substitute for an inventory, calendar or ticketing tool.

Standard procedures
- Stock receipt: when a delivery is logged with receive_shipment, the on-hand quantity rises by the received units and the result reports the new on-hand quantity. No separate confirmation call is needed.
- Order fulfilment: ship_order fails with an insufficient-stock error when the requested units exceed the on-hand quantity; report the error and do not partially ship.

```
- Price changes: set_price takes effect
↪  immediately in the storefront. Quote
↪  prices with two decimals when the
↪  coordinator asks for them.
- Relocation: move_stock relocates every
↪  unit of a SKU; there are no partial
↪  moves. list_bin_contents reflects the
↪  bin's contents as of the call.
- Rescheduling: reschedule_event keeps the
↪  event on the same date; a move to
↪  another date requires cancelling and
↪  recreating, which this console does not
↪  support. move_event_room does not check
↪  for room conflicts; get_room_schedule
↪  does.
- Attendance: add_attendee and
↪  remove_attendee return the complete
↪  attendee list after the change; the
↪  number of attendees is the length of
↪  that list.
- Triage: assign_ticket replaces the
↪  previous assignee outright. set_priority
↪  sets an absolute level; escalate_ticket
↪  raises by exactly one level and never
↪  above P1. set_status accepts only open,
↪  pending, in_progress and resolved.
- Reporting: when a question asks for a
↪  current value, the most recent result
↪  for that record and field in this
↪  conversation is the answer. Do not
↪  average, do not combine old and new
↪  values, and do not assume a change you
↪  did not see confirmed by a tool result.
Glossary
- On-hand quantity: units physically present
↪  in the main warehouse, excluding inbound
↪  shipments not yet received and outbound
↪  orders not yet shipped.
- Reorder threshold: the on-hand level at or
↪  below which purchasing raises an order;
↪  it is set per SKU and changes only
↪  through set_reorder_threshold.
- Bin: a shelf location code (letter,
↪  hyphen, two digits). A SKU occupies
↪  exactly one bin at a time.
- Lead time: supplier-quoted days from
↪  purchase order to delivery;
↪  informational only.
- Event status: confirmed events appear on
↪  room schedules; cancelled events do not,
↪  but their record remains readable with
↪  get_event.
- Attendee list: the people invited to an
↪  event, in invitation order; the
↪  organiser is included as an attendee.
- Ticket status: open (not yet picked up),
↪  pending (waiting on the customer),
↪  in_progress (being worked), resolved
↪  (closed; excluded from agent load).
- Agent load: the number of unresolved
↪  tickets assigned to one agent, as
↪  returned by get_agent_load at the time
↪  of the call.
- Response SLA: the policy response time in
↪  hours for a priority level; it is a
↪  property of the priority, not of the
↪  ticket.
- Superseded observation: an earlier tool
↪  result whose value for a record and
↪  field has been replaced by a later tool
↪  result; it must not be used to answer a
↪  question about the current value.
- Consistency. Treat the coordinator's
↪  instructions as authoritative about what
↪  to do and the tool results as
↪  authoritative about what is true. If an
↪  instruction refers to a record you have
↪  not seen, call the tool anyway; the
↪  backend will validate the identifier.
```

Tool catalogue

The 39 tool declarations (names, descriptions and JSON schemas) ship in `corpus/tools.json`; the names are: `get_stock`, `get_price`, `get_reorder_threshold`, `get_supplier`, `list_bin_contents`, `receive_shipment`, `ship_order`, `set_price`, `move_stock`, `set_reorder_threshold`, `get_event`, `get_room_schedule`, `get_availability`, `reschedule_event`, `move_event_room`, `add_attendee`, `remove_attendee`, `cancel_event`, `set_event_title`, `get_ticket`, `get_agent_load`, `get_sla_policy`, `list_tickets`, `assign_ticket`, `set_priority`, `set_status`, `add_tag`, `add_comment`, `escalate_ticket`, `send_message`, `post_channel_update`, `schedule_message`, `cancel_scheduled_message`, `get_channel_members`, `get_employee`, `get_time_off_balance`, `request_time_off`, `submit_expense`, `post_job_opening`.

User-turn phrasings

Every action instruction and every question has two phrasings; phrasing 1 is the main grid. Examples (phrasing 1 / phrasing 2):

```
get_stock: Check the current stock level of
↪  {sku}.  /  How many {sku} do we have on
↪  hand right now? Look it up.
receive_shipment: Record that we received
↪  {units} units of {sku}.  /  A delivery
↪  of {units} units of {sku} just arrived.
↪  Log the receipt.
list_tickets: List the tickets currently in
↪  status {status}.  /  Show me everything
↪  that's {status} right now.
reschedule_event: Reschedule event
↪  {event_id} to {new_start}-{new_end}.  /
↪  Move {event_id} so it runs {new_start}
↪  to {new_end}.
```

```
question wrapper 1: Based only on the
↪  information already gathered in this
↪  conversation, and without calling any
↪  tools (tool calling is disabled for this
↪  question): {q} Reply with JSON
↪  {{"answer": {fmt}}} and nothing else.
question wrapper 2: No tool calls for this
↪  one, just use what we have already seen
↪  in this conversation (tools are disabled
↪  here): {q} Reply with JSON {{"answer":
↪  {fmt}}} only.

qty: what is the current on-hand quantity of
↪  {record}?  (format <number>)
start: at what time does event {record}
↪  currently start?  (format <HH:MM>)
assignee: who is currently assigned to
↪  ticket {record}?  (format <full name>)
below_threshold: is {record} currently at or
↪  below its reorder threshold?  (format
↪  yes or no)
```

L4 session note (template)

```
[Session note before step 9]
Goal: <goal sentence>
Rule: for every record use the latest
↪  observation listed as current;
↪  observations listed as superseded are no
↪  longer valid.
Records with updated observations:
- <record>: current <field> at step <k>;
↪  superseded: steps <i>, <j>
- <n> other records observed once; those
↪  observations are current.
[End of note]
```

E Statistical test family

The family holds 65 tests; 29 survive Benjamini-Hochberg at $q = 0.05$ (effective threshold $p \leq 0.0207$). Every test in Table 6 is a two-sided sign-flip permutation over tasks (20,000 flips) unless its name says `int` (label permutation, 10,000) or `battery`. Table 7 gives the interaction estimates.

Model	Test	p	BH	Model	Test	p	BH
Gemini 2.5 Flash	cache_frac:L1-L3	0.581		Gemini 3.1 Flash-Lite	int:L1-L2:trap_vs_neutral	0.111	
Gemini 2.5 Flash	cache_frac:L4-L1	0.030		Gemini 3.1 Flash-Lite	int:L1-L3:distractor	0.538	
Gemini 2.5 Flash	cache_frac:L4-L3	0.112		Gemini 3.1 Flash-Lite	int:L1-L3:length	0.847	
Gemini 2.5 Flash	cost:L1-L3	0.021	✓	Gemini 3.1 Flash-Lite	int:L1-L3:trap_vs_neutral	<0.0001	✓
Gemini 2.5 Flash	cost:L4-L1	<0.0001	✓	Gemini 3.1 Flash-Lite	int:L4-L1:distractor	0.719	
Gemini 2.5 Flash	cost:L4-L3	<0.0001	✓	Gemini 3.1 Flash-Lite	int:L4-L1:length	0.011	✓
Gemini 2.5 Flash	int:L1-L3:distractor	0.377		Gemini 3.1 Flash-Lite	int:L4-L1:trap_vs_neutral	0.0014	✓
Gemini 2.5 Flash	int:L1-L3:length	0.355		Gemini 3.1 Flash-Lite	neutral_acc:L1-L2	1.000	
Gemini 2.5 Flash	int:L1-L3:trap_vs_neutral	<0.0001	✓	Gemini 3.1 Flash-Lite	neutral_acc:L1-L3	1.000	
Gemini 2.5 Flash	int:L4-L1:distractor	0.196		Gemini 3.1 Flash-Lite	neutral_acc:L2-L3	1.000	
Gemini 2.5 Flash	int:L4-L1:length	0.559		Gemini 3.1 Flash-Lite	neutral_acc:L4-L1	1.000	
Gemini 2.5 Flash	int:L4-L1:trap_vs_neutral	<0.0001	✓	Gemini 3.1 Flash-Lite	neutral_acc:L4-L3	1.000	
Gemini 2.5 Flash	neutral_acc:L1-L3	1.000		Gemini 3.1 Flash-Lite	success:L1-L2	0.334	
Gemini 2.5 Flash	neutral_acc:L4-L1	1.000		Gemini 3.1 Flash-Lite	success:L1-L3	<0.0001	✓
Gemini 2.5 Flash	neutral_acc:L4-L3	1.000		Gemini 3.1 Flash-Lite	success:L2-L3	<0.0001	✓
Gemini 2.5 Flash	success:L1-L3	<0.0001	✓	Gemini 3.1 Flash-Lite	success:L4-L1	0.034	
Gemini 2.5 Flash	success:L4-L1	<0.0001	✓	Gemini 3.1 Flash-Lite	success:L4-L3	<0.0001	✓
Gemini 2.5 Flash	success:L4-L3	0.147		Gemini 3.1 Flash-Lite	trap_acc:L1-L2	0.116	
Gemini 2.5 Flash	trap_acc:L1-L3	<0.0001	✓	Gemini 3.1 Flash-Lite	trap_acc:L1-L3	<0.0001	✓
Gemini 2.5 Flash	trap_acc:L4-L1	<0.0001	✓	Gemini 3.1 Flash-Lite	trap_acc:L2-L3	<0.0001	✓
Gemini 2.5 Flash	trap_acc:L4-L3	0.043		Gemini 3.1 Flash-Lite	trap_acc:L4-L1	0.0013	✓
Gemini 3.1 Flash-Lite	cache_frac:L1-L2	<0.0001	✓	Gemini 3.1 Flash-Lite	trap_acc:L4-L3	<0.0001	✓
Gemini 3.1 Flash-Lite	cache_frac:L1-L3	<0.0001	✓	Gemini 3.6 Flash	cache_frac:L1-L3	0.351	
Gemini 3.1 Flash-Lite	cache_frac:L2-L3	<0.0001	✓	Gemini 3.6 Flash	cost:L1-L3	0.196	
Gemini 3.1 Flash-Lite	cache_frac:L4-L1	0.642		Gemini 3.6 Flash	int:L1-L3:distractor	0.597	
Gemini 3.1 Flash-Lite	cache_frac:L4-L3	<0.0001	✓	Gemini 3.6 Flash	int:L1-L3:length	1.000	
Gemini 3.1 Flash-Lite	cost:L1-L2	<0.0001	✓	Gemini 3.6 Flash	int:L1-L3:trap_vs_neutral	0.016	✓
Gemini 3.1 Flash-Lite	cost:L1-L3	<0.0001	✓	Gemini 3.6 Flash	neutral_acc:L1-L3	1.000	
Gemini 3.1 Flash-Lite	cost:L2-L3	<0.0001	✓	Gemini 3.6 Flash	success:L1-L3	0.126	
Gemini 3.1 Flash-Lite	cost:L4-L1	<0.0001	✓	Gemini 3.6 Flash	trap_acc:L1-L3	0.030	
Gemini 3.1 Flash-Lite	cost:L4-L3	<0.0001	✓	family	int:L1-L3	0.035	
Gemini 3.1 Flash-Lite	int:L1-L2:distractor	1.000		family	int:L1-L3:common	0.092	
Gemini 3.1 Flash-Lite	int:L1-L2:length	0.643					

Table 6: The full test family: every test with its two-sided p. ✓: survives Benjamini-Hochberg at $q = 0.05$.

Model	Contrast	Gap slope per call (p)	High − low distractor (p)	Trap − neutral gap (p)
Gemini 2.5 Flash	L1 − L3 (content)	+0.0049 (0.355)	+0.083 (0.377)	**-0.273 (<0.0001)**
	L4 − L1 (note)	-0.0033 (0.559)	-0.125 (0.196)	**+0.215 (<0.0001)**
Gemini 3.1 Flash-Lite	L1 − L2 (order)	+0.0012 (0.643)	+0.000 (1.000)	-0.036 (0.111)
	L1 − L3 (content)	+0.0006 (0.847)	-0.035 (0.538)	**-0.177 (<0.0001)**
	L4 − L1 (note)	-0.0065 (0.011)	+0.017 (0.719)	**+0.073 (0.0014)**
Gemini 3.6 Flash	L1 − L3 (content)	-0.0000 (1.000)	+0.074 (0.597)	**-0.139 (0.016)**

Table 7: Interaction tests by label permutation. The gap slope is the change in the trap-accuracy contrast per additional tool call in the episode. The trap-minus-neutral column is the difference of differences; bold survives BH.

F Per-cell tables

Model	Layout	6 calls	12	18	30	low distr.	high distr.
Gemini 2.5 Flash	L1 append-only	0.611	0.694	0.722	0.611	0.630	0.690
	L3 pruned	1.000	0.944	0.926	0.861	0.944	0.921
	L4 hybrid	0.889	0.917	0.889	0.806	0.907	0.843
Gemini 3.1 Flash-Lite	L1 append-only	0.778	0.778	0.917	0.750	0.817	0.794
	L2 reordered	0.861	0.819	0.880	0.806	0.853	0.830
	L3 pruned	1.000	0.986	0.972	0.972	0.977	0.988
	L4 hybrid	0.944	0.903	0.898	0.771	0.882	0.876
Gemini 3.6 Flash	L1 append-only	0.889	–	0.889	–	0.852	0.926
	L3 pruned	1.000	–	1.000	–	1.000	1.000

Table 8: Trap accuracy by episode length (tool calls) and by distractor level, per model and layout. Each length cell holds 36 episodes for the full grid.

Model	Layout	own	external	chain 1	chain 2	chain 3	inventory	calendar	tickets
Gemini 2.5 Flash	L1 append-only	0.988	0.400	0.733	0.597	0.556	0.691	0.677	0.611
	L3 pruned	1.000	0.840	0.956	0.889	0.778	0.910	0.969	0.920
	L4 hybrid	1.000	0.750	0.889	0.861	0.722	0.778	0.944	0.903
Gemini 3.1 Flash-Lite	L1 append-only	1.000	0.655	0.850	0.771	0.750	0.804	0.788	0.825
	L2 reordered	1.000	0.705	0.867	0.806	0.806	0.861	0.844	0.819
	L3 pruned	1.000	0.960	1.000	0.951	0.972	0.995	0.976	0.977
	L4 hybrid	1.000	0.735	0.922	0.792	0.750	0.887	0.899	0.851
Gemini 3.6 Flash	L1 append-only	1.000	0.800	0.889	0.889	–	0.972	0.722	0.972
	L3 pruned	1.000	1.000	1.000	1.000	–	1.000	1.000	1.000

Table 9: Trap accuracy by trap kind (own update versus external update revealed in a listing), by chain length (number of updates before the question) and by domain.

Model	Layout	Action exec.	Reveal exec.	Trap acc. (all)	Trap acc. (thread fully executed)	n skipped
Gemini 2.5 Flash	L1 append-only	0.997	1.000	0.661	0.661	0
	L3 pruned	0.997	1.000	0.911	0.911	0
	L4 hybrid	1.000	1.000	0.861	0.861	0
Gemini 3.1 Flash-Lite	L1 append-only	0.998	0.995	0.808	0.811	1
	L2 reordered	0.998	1.000	0.836	0.836	0
	L3 pruned	0.996	0.990	0.978	0.983	2
	L4 hybrid	0.995	0.990	0.853	0.858	2
Gemini 3.6 Flash	L1 append-only	0.984	0.975	0.889	0.884	3
	L3 pruned	0.988	1.000	1.000	1.000	2

Table 10: Action-fidelity sensitivity. "Reveal exec." is the share of scripted reveal steps the agent actually executed with the right arguments. The last two columns restrict trap accuracy to questions whose thread steps were all executed; "skipped" counts questions where a thread step was not executed, so the fresh value never entered the context.

Model	L3 − L1 gap	L4 − L1 gap	Recovery [CI]	L4/L1 cached share	Cheapest per correct (billed)
Gemini 2.5 Flash	+0.273	+0.215	0.79 [0.58, 0.97]	0.95	L3
Gemini 3.1 Flash-Lite	+0.177	+0.073	0.42 [0.21, 0.60]	1.00	L3

Table 11: D3 and D5 summary per model. Recovery is (L4 − L1)/(L3 − L1) on trap accuracy with a task-bootstrap interval.

Model	0.0×	0.01×	0.025×	0.05×	0.1×	0.25×	0.5×	0.75×	1.0×
Gemini 2.5 Flash	L3	L3	L3	L3	L3	L3	L3	L3	L3
Gemini 3.1 Flash-Lite	L3	L3	L3	L3	L3	L3	L3	L3	L3
Gemini 3.6 Flash	L3	L3	L3	L3	L3	L3	L3	L3	L3

Table 12: Cheapest layout per correct task as the cache-read price moves from free (0×) to the full input price (1×); the billed point for Google sits at 0.1×.

G Batteries

Repeat battery. 36 task cells on L1 were re-run (five runs each, temperature 0). The trap-accuracy outcome differed between runs in 11.1 percent of cells and episode success in 8.3 percent; the mean within-cell standard deviation of question accuracy is 0.017.

Phrasing battery (Gemini 3.1 Flash-Lite). 36 episodes were re-run with the second phrasing of every instruction and question. Question accuracy under phrasing 2 minus phrasing 1: -0.007 [-0.042, 0.032], sign-flip $p = 0.879$.

Cache on/off. Not run: the only provider that documents output-neutral explicit caching (Anthropic) was unreachable on this workspace's OpenRouter key (a guardrail that blocks every non-Google model; probed before the plan was frozen and again after the main grid). The implicit-cache hit-versus-miss comparison within layout on Google is reported in its place.

H Hand audit

Sample: analyze.py –audit-export 40: wrong trap answers from the main gemini-3.1-flash-lite grid, stratified across layouts (L1 11, L2 11, L3 8 = all L3 wrong answers, L4 10), seeded. Wrong trap answers in the main grid: 189. Coder: Claude Fable 5.1 (the agent that ran the study), coded from results/_audit_view_0_40.txt which lists every observation of the queried field before the question; not blind to layout. Date: 2026-10-10.

Code	L1	L2	L3	L4	All
`stale_value`	11	9	5	9	34
`ambiguous_stale_or_boundary`	0	2	0	1	3
`reveal_not_executed`	0	0	2	0	2
`arithmetic_or_count`	0	0	1	0	1
total	11	11	8	10	40

Table 13: Hand-coded wrong trap answers by layout. The per-item codes and notes ship in `results/audit_sample.json`.

`stale_value` the answer equals an earlier observation of the same (record, field) that a later tool result had superseded

`reveal_not_executed` the agent skipped the scripted tool call that would have revealed the fresh value, so the fresh value never entered the context (an action-fidelity failure, not a stale read)

`ambiguous_stale_or_boundary` below-threshold question where the fresh quantity equals the threshold; the wrong answer fits both a stale read and a strict-inequality misreading

`arithmetic_or_count` the answer matches no observed value and looks derived

`distractor_record` the answer equals another record's value for the same field (none found as the sole explanation)

`abstain_or_unparsed` no parseable answer (none in sample)

I Cost and compute

Every call was billed through OpenRouter and logged by a locked cost tracker with a hard stop at USD 24. The total below includes pilots, calibration episodes, smoke tests, discarded runs and the external arm; the main grids are the `gemini-*` rows. No GPU was used; the executor, layouts and analysis run on a laptop in under a minute.

Item (OpenRouter billed)	USD
google/gemini-3.1-flash-lite	11.82
google/gemini-2.5-flash	3.90
google/gemini-3.6-flash	2.55
google/gemini-2.5-flash-lite	0.12
google/gemini-3.5-flash-lite	0.03
_smoke_and_diag_2026-10-10	0.01
total (24817 calls)	18.42

Table 14: Metered spend by model and arm (USD, OpenRouter `usage.cost`).